Managing a Python project and its *Python Virtual Environment* (PVE) with UV

Learning outcomes:
- Learn how to manage a Python project and its Python Virtual Environment (PVE) using the project manager **uv**.

Expected duration:
- 30 minutes (depending on your Internet connection).

▸ Interest

The state of the art in Python programming (Data processing, Machine Learning…) is to develop each project within a **Python Virtual Environment** (PVE) which provides a dedicated and persistent environment containing a specific installation of Python:

- independent of other Python installations likely to coexist on the same machine,
- independent of computer updates.

A PVE is based on a dedicated disk tree that houses the desired version of the Python interpreter and the specific versions of the Python modules needed for the project. You can create, duplicate, delete and recreate a PVE very easily, without impacting other Python installations possibly present on your computer.

▸ Tools

The most often used tools until know to create PVE were:

- the `conda` command, available if you have installed miniconda or Anaconda on your computer
- the `venv` Python module (see venv)
- the `poetry` tool (see poetry.org).

Recently a new tool has been unveiled to manage Python project within a PVE : uv.

It is considerably faster and easier to use than the previous tools: so let's see how to use **uv** for our Python project.

▸ How to install **uv** on your computer

▸ 1 – Windows

Type the word **`powershell`** in the *Windows search bar* and clic on 

to run a **Windows PowerShell**, then copy/paste the command bellow in the PowerShell window and hit ENTER:

```
powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/install.ps1 | iex"
```

▸ 2 – Mac OS & GNU/Linux

Open a **terminal window**, then copy/paste the command bellow and hit ENTER:

```
curl -LsSf https://astral.sh/uv/install.sh | sh
```

If you don't have `curl` installed on your system, then you can use `wget` as shown below:

```
wget -qO- https://astral.sh/uv/install.sh | sh
```

▸ Create the project directory and the PVE

▸ 1 – Get the "AI-ML_at_ENSPIMA" GitHub repository

- Open the Git repository https://github.com/cjlux/AI-ML_at_ENSPIMA.
- Download the ZIP archive with the button 

.
- Extract the directory **AI-ML_at_ENSPIMA-master** from the ZIP archive somewhere in your working tree.
- With the file manager, copy the path of the directory **AI-ML_at_ENSPIMA-master**: we will denote this path in the rest of the document as **<path of AI-ML_at_ENSPIMA>**.

▸ 2 – Create the PVE with **uv**

Open a new console (Windows: a new *powershell* window, Mac/Linux: a new *terminal*) and go to the directory of the project with the `cd` command (*change directory*):

```
cd <path of AI-ML_at_ENSPIMA>
```

- Create a PVE with the Python 3.12 interpreter in the **AI-ML_at_ENSPIMA-master** project directory:

```
uv init -p 3.12
```

▸ 3 – Install the Python modules within the PVE

Now that the PVE has been created inside the project directory, `uv` will automatically install the Python modules in this PVE if you use the '`uv add …`' command inside the project directory:

- Check that your working directory is indeed the project directory with the pwd command (*print working directory*)

```
pwd
```

if the working directory is not <path of AI-ML_at_ENSPIMA> go to the wright directory with the cd command (*change directory*):

```
cd <path of AI-ML_at_ENSPIMA>
```

- Now, install the version 2.17 of tensorflow with the 'uv add' command:

```
uv add tensorflow==2.17
```

▸ Windows only

To fully install the tensorflow module under Windows you must also type the command:

```
uv pip install tensorflow==2.17
```

- Continue to add some Python modules useful for your project:

```
uv add jupyterlab matplotlib opencv-python pandas scikit-learn seaborn
```

Tips:
▸ All the Python modules are installed in the **.venv** subdirectory of **AI-ML_at_ENSPIMA-master**.

▸ You can look at the **pyproject.toml** file to see how uv keeps track of the modules installed with the 'uv add' command.

▸ 3 – Work within the project PVE

- Once the PVE is installed with its Python `modules` in the project directory, you can run the jupyter IDE within the PVE using the prefix 'uv run':

```
uv run jupyter lab
```

- You can also run the Python interpreter within the PVE using the 'uv run' prefix:

```
uv run python
```

- If you need to run a Python programm within the PVE use the 'uv run' prefix:

```
uv run python <path to any Python program>
```

▶ Useful uv commands

command	description
uv pip list	Lists the python packages of the virtual environment.
uv sync	Installs all the Python packages as listed in the **pyproject.toml** file.
uv self update	Updates uv to the last bersion.